

ML_Lab5_Decision_Tree

September 22, 2026

Decision Tree Classification

```
[ ]: import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
import matplotlib.pyplot as plt

[ ]: # Step 1: Creating the Dataset
data = {
    "Height": [170, 170, 185, 155, 175],
    "Weight": [75, 90, 85, 50, 78],
    "Gender": ["Female", "Male", "Male", "Female", "Female"]
}

[ ]: df = pd.DataFrame(data)

[ ]: # Step 2: Encoding the Target Variable (Gender)
df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})

[ ]: # Step 3: Splitting Features (X) and Target Variable (y)
X = df[["Height", "Weight"]]
y = df["Gender"]

[ ]: # Step 4: Training the Decision Tree Classifier
model = DecisionTreeClassifier(criterion="entropy", max_depth=3,
    ↪random_state=42)
model.fit(X, y)

[ ]: # Step 5: Printing Decision Rules
tree_rules = export_text(model, feature_names=X.columns.to_list())
print("\nDecision Tree Rules:\n")
print(tree_rules)

[ ]: # Step 6: Visualizing the Decision Tree
plt.figure(figsize=(8, 6))
plot_tree(model, feature_names=X.columns, class_names=["Female", "Male"],
    ↪filled=True)
plt.show()
```

```
[ ]: df = pd.read_csv("salaries.csv")
df

[ ]: inputs = df.drop('salary_more_than_100k',axis='columns')

[ ]: target = df['salary_more_than_100k']

[ ]: from sklearn.preprocessing import LabelEncoder
le_company = LabelEncoder()
le_job = LabelEncoder()
le_degree = LabelEncoder()

[ ]: inputs['company_n'] = le_company.fit_transform(inputs['company'])
inputs['job_n'] = le_job.fit_transform(inputs['job'])
inputs['degree_n'] = le_degree.fit_transform(inputs['degree'])

[ ]: inputs

[ ]: inputs_n = inputs.drop(['company', 'job', 'degree'],axis='columns')

[ ]: X = inputs_n

[ ]: Y = target

[ ]: X.shape

[ ]: #Split data into training and test sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size = 0.1)

[ ]: from sklearn import tree
model = tree.DecisionTreeClassifier()

[ ]: model.fit(inputs_n, target)

[ ]: y_pred = model.predict(X_test)
X_train.shape

[ ]: import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=["company_n", "job_n", "degree_n"],
class_names=["<=100k", ">100k"], filled=True)
plt.show()
```

```
[ ]: from sklearn.metrics import classification_report, confusion_matrix, \
      ↪accuracy_score

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
cr = classification_report(y_test, y_pred)
print("\nClassification Report:",)
print (cr)
acc = accuracy_score(y_test,y_pred)
print("\nAccuracy:",acc)

[ ]: import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Sample Data
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, \
      ↪random_state=42)

# Train a Decision Tree Model
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)

from sklearn.tree import DecisionTreeClassifier, plot_tree

# Sample Data
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, \
      ↪random_state=42)

# Train a Decision Tree Model
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)

# Plot the Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(clf, feature_names=iris.feature_names, class_names=iris.target_names, \
      ↪filled=True)
plt.show()
```

```
print(iris.feature_names)
```

Is salary of Google, Computer Engineer, Bachelors degree > 100 k ?

```
[ ]: model.predict([[2,1,0]])
```

Is salary of Google, Computer Engineer, Masters degree > 100 k ?

```
[ ]: model.predict([[2,1,1]])
```

Exercise: Build decision tree model to predict survival based on certain parameters

0.1 Lab activity:

Task 1: The Titanic Survival dataset is a widely-used dataset in the field of machine learning and data analysis. It contains information about the passengers on board the Titanic, including whether they survived the shipwreck or not. ##### In titanic.csv file, using following columns build a model to predict if person would survive or not,

1. Pclass
2. Sex
3. Age
4. Fare

Generate classification report, confusion matrix and accuracy on the test set (you may consider 30% of the data for testing)

Task 2: Using the Titanic Survival dataset, train a DecisionTreeClassifier and investigate the effect of the following hyperparameters:

criterion: gini, entropy max_depth: 1, 2, 3, 4, 5

Use 5-Fold Cross-Validation to identify the combination of criterion and max_depth that gives the best cross-validation accuracy.

After identifying the optimal hyperparameters:

Train the final Decision Tree using the selected hyperparameters. Evaluate its performance on the test set. Display the resulting Decision Tree. Compare the cross-validation accuracy with the test accuracy.

```
[ ]:
```

```
[ ]:
```